# Expense Tracker on AWS: Project Brief (As Built)

Nov 5, 2026 · @Cesar Nignan

The serverless Expense Tracker is live on a public HTTPS CloudFront URL. Users sign up and log in through Cognito, and their expenses are stored per user in DynamoDB behind a protected API. Below, each step of the brief's AWS Implementation Sequence is listed with what we did, why, and what the service does.

## Architecture at a glance

The site is static files, so the browser handles all the screens and only talks to AWS for login and for expense data. Nothing runs on a server we manage.

1. The user opens the CloudFront HTTPS URL. CloudFront (with AWS WAF in front) serves the HTML, CSS and JavaScript from a private S3 bucket.
2. The user clicks Log in or Sign up and lands on Cognito's Hosted UI. Cognito verifies the email and password and sends the browser back with a one-time code.
3. The frontend exchanges that code for tokens (using PKCE) and keeps them in the browser.
4. For every expense action, the frontend calls API Gateway with the access token in the Authorization header.
5. API Gateway's Cognito JWT authorizer validates the token. Only then does it invoke the Lambda function.
6. Lambda reads the user's Cognito `sub` from the validated token and reads or writes only that user's rows in DynamoDB.
7. Lambda logs to CloudWatch, which is where we troubleshoot.

Region for everything: London (eu-west-2).

## Who owns what: infrastructure vs. generated code

The application code (the website's HTML, CSS and JavaScript, and the Python Lambda function) was generated with AI assistance and then tested by the team. That code is the *payload*: it decides how the screens look, how a form is validated and how an expense is

saved or read. It is not where our role as cloud and DevOps engineers lies. Our responsibility is the platform that runs, protects and delivers that code.

| Layer | What it covers | Who is responsible |
|---|---|---|
| Application code | Page layouts, form checks, the logic that builds and reads expense records, error messages | AI-generated code, reviewed and tested by the team |
| Cloud infrastructure | DynamoDB table and keys, Cognito pool and app client, Lambda deployment and runtime, API Gateway routes, S3, CloudFront, WAF | Us, the cloud and DevOps engineers |
| Security and access | IAM roles with least privilege, JWT authorizer, CORS, private bucket with OAC, HTTPS, no secrets in GitHub | Us |
| Operations | Configuration values, deployment and cache invalidation, CloudWatch logs, troubleshooting, cost control, documentation | Us |

**Why this split matters:**

- The brief's deliverables are infrastructure ones: a database, a protected API, a deployed backend, a public HTTPS site, architecture documentation and screenshots. They show that we can design, connect, secure and operate AWS services.
- Most of the problems we actually solved were infrastructure problems, not code problems: the app client's secret, the callback URL, CORS and the authorizer audience. Fixing them took cloud knowledge, not programming.
- A working app with misconfigured services is still a failed deployment, while a simple app on correctly secured infrastructure is a success.

**Our limit of responsibility:** we don't need to write the application code, but we do need to understand what it expects from the platform (for example, that Lambda reads the user ID from the authorizer, or that the frontend needs three config values). That understanding is what lets us deploy it correctly and debug it. If your programme asks for it, state openly in the presentation that the application code was AI-generated.

## Step 1: AWS account and access

**What we did:** chose the AWS account that hosts the project, gave each member their own authorized login, settled on London (eu-west-2) as the region, and agreed on resource names (the DynamoDB table is `Expenses`). Set a billing budget or alert if one isn't in place yet.

**Why:** everything else is built inside this account, so we fix who can touch it, where it lives and what it may cost before creating anything. One region keeps every service in the same place, which avoids cross-region mistakes and extra latency.

**Role of the service:** AWS IAM (identity and access management) controls who and what may act on which resources. Individual logins mean every change is traceable to a person. AWS Budgets warns us before the bill surprises us.

**In depth:**

- Cloud resources are created and billed inside one AWS account, so account-level decisions come first. Using one named login per person (never a shared root login) gives an audit trail in CloudTrail: you can see who changed what.
- Multi-factor authentication on every login is the single most effective protection for the account.
- Choosing the region first matters because most services are regional. Cognito, the API, Lambda and DynamoDB must be in the same region (eu-west-2) to talk to each other simply. CloudFront and WAF for CloudFront are the exceptions, since they are global (WAF for CloudFront must be created in us-east-1).
- Naming conventions (for example `expense-tracker-*`) make resources easy to find, document and clean up.
- A budget alert is the safety net against a forgotten resource or a traffic spike running up a bill.

## Step 2: DynamoDB table

**What we did:** created the `Expenses` table with `userId` (String) as partition key and `expenseId` (String) as sort key, on-demand billing, then tested creating and reading records.

| Attribute | Type | Meaning |
|---|---|---|
| userId | String | Cognito `sub` of the owner |
| expenseId | String | Unique ID generated by Lambda (UUID) |
| amount | Number | Amount spent |
| category | String | Food, Transport, Bills, Shopping, Entertainment or Other |
| description | String | What it was for (100 characters max) |
| expenseDate | String | Date of the expense, YYYY-MM-DD |
| createdAt | String | Timestamp when it was saved |

**Why this key design:** the partition key groups all of one user's expenses together, so "get this user's expenses" is a single fast query and nobody else's rows come back. The sort key makes each expense unique within that user. On-demand billing means no capacity to size, and we pay only for what we use, which suits a small project with unpredictable traffic.

**Role of the service:** Amazon DynamoDB is the serverless NoSQL database that stores the expense records. It scales automatically and needs no servers or patching. Passwords are never stored here; Cognito owns them.

**In depth:**

- DynamoDB stores items found by key. A query with `userId = X` reads only that partition, so cost and speed don't grow with the number of other users.
- Numbers are stored as DynamoDB `Number` (exact decimals), which is why Lambda converts them back to normal JSON numbers when responding.
- We chose Strings for both keys because Cognito's `sub` and a UUID are strings.
- `expenseDate` is a plain text date in `YYYY-MM-DD` form, which sorts correctly as text. For this first version the app sorts by that date, as the brief says, rather than using a more complex index.
- Testing at this stage means creating an item by hand in the console and reading it back, to prove the table and keys work before any code depends on them.
- Nothing here is public. Access is only possible through IAM permissions, which is why Step 4 matters.

## Step 3: Cognito user pool and app client

**What we did:** created a User Pool with email as the sign-in identifier, email verification turned on, password sign-in enabled, `name` as a required attribute and a sensible password policy (8+ characters with upper, lower, number and special). Then we created an app client for the website and turned on the Hosted UI (Cognito's managed login and signup pages) with a Cognito domain.

**App client settings that matter:**

- Type: single-page application, **no client secret** (a browser cannot keep a secret safely).
- Flow: Authorization code grant with PKCE. Scopes: `openid`, `email`, `profile`.
- Callback and sign-out URLs: the CloudFront `index.html` URL, plus `http://localhost:5500/index.html` for local testing.

**Why:** the brief says Cognito must handle passwords and authentication, never our own code or database. Using the Hosted UI means we write no login, signup, verification or reset pages, and Cognito's tested screens handle all of that, including the emailed verification and reset codes.

**Role of the service:** Amazon Cognito registers users, verifies their emails, checks passwords and issues signed tokens (ID, access and refresh). The access token is what proves to API Gateway who the caller is. The `sub` claim inside it is each user's permanent unique ID, which we use as `userId`.

**In depth:**

- A **user pool** is the user directory. Signing up creates a user there, sends a verification code by email, and only a confirmed user can log in.
- The **Hosted UI** is a login website that Cognito runs for us on our own Cognito domain. The frontend only redirects to it, so the user types the password into Cognito, never into our code.
- **Authorization code flow with PKCE:** after login, Cognito returns a short-lived code, not tokens. The browser then trades the code, plus a secret it generated itself (the PKCE verifier), for tokens. This stops anyone who intercepts the code from using it.
- Three tokens come back: the **ID token** (who the user is, such as name and email), the **access token** (what the API accepts) and the **refresh token** (gets new tokens without logging in again).

- The callback URL list is a safety rule: Cognito only sends users back to addresses we registered. That is why the CloudFront URL had to be added after it existed.
- A **public** app client has no secret. A client with a secret is meant for servers, and using one from a browser fails with `invalid_client`, which is what we hit and fixed.

## Step 4: Lambda function and IAM role

**What we did:** created one Python 3.12 Lambda function that handles all three expense routes by looking at the HTTP method. Its execution role was limited to the DynamoDB actions it needs ( `PutItem`, `Query`, `DeleteItem`) on the `Expenses` table. We tested it with sample events before connecting anything to it.

| Route | What it does |
|---|---|
| GET /expenses | Returns the user's expenses (newest first) and the total |
| POST /expenses | Validates and saves a new expense (amount above 0, one of the 6 categories, description up to 100 characters, date as YYYY-MM-DD) |
| DELETE /expenses/{expenseId} | Deletes that expense, or returns 404 if it isn't the caller's |

**Why:** the logic sits on the server because a browser can't be trusted. Lambda reads `userId` only from the validated token ( `requestContext.authorizer.jwt.claims.sub`) and never from the request body, so a user cannot read, add or delete another user's data even by editing requests. The narrow IAM role means that even a bug in the code can't touch other tables or other actions (least privilege).

**Role of the services:** AWS Lambda runs our Python code only when a request arrives, with no server to manage, and bills per use. IAM gives the function its permissions. CloudWatch automatically receives its logs, so errors are visible without exposing details to users (they only get a generic message).

**In depth:**

- A Lambda function is code plus a configuration: runtime (Python 3.12), handler name, memory, timeout and an **execution role**. The role is an IAM identity the function assumes when it runs.

- Least privilege means the role allows only `PutItem`, `Query` and `DeleteItem`, on this one table's ARN, plus permission to write logs. It cannot scan the table, touch other tables or use other AWS services.
- The table name comes from an environment variable (`TABLE_NAME`, defaulting to `Expenses`), so the code isn't tied to one environment.
- API Gateway sends each request to Lambda as an *event* (a JSON document). Testing with sample events means pasting such a JSON in the Lambda console, including a fake `requestContext` with a `sub`, to check each route before the real API exists.
- Every execution writes to a CloudWatch log group automatically, where print and error output can be read.
- Code deployment: the Python file is uploaded as the function's code and the **Deploy** button publishes it. This is an infrastructure task even though the code itself was generated.

## Step 5: API Gateway, JWT authorizer and CORS

**What we did:** created an HTTP API and added three routes (`GET /expenses`, `POST /expenses`, `DELETE /expenses/{expenseId}`), all integrated with the Lambda function. We attached a Cognito JWT authorizer to every route (issuer: the user pool in eu-west-2; audience: the app client ID). CORS allows only the CloudFront URL and `http://localhost:5500`, with the headers `authorization` and `content-type` and the methods `GET`, `POST`, `DELETE` and `OPTIONS`.

**Why:** API Gateway is the single front door to the backend. The authorizer rejects any request without a valid, unexpired Cognito token before Lambda even runs, so unauthenticated callers can never reach our code or our data. CORS restricts which websites a browser will let call the API, so a random site can't use a visitor's browser against it.

**Role of the service:** Amazon API Gateway exposes the backend as HTTPS endpoints, connects each route to Lambda, and enforces authentication. An HTTP API is the simpler, cheaper type and supports the JWT authorizer directly. The authorizer also passes the verified token claims to Lambda, which is how Lambda learns the user's `sub` without trusting the frontend.

**In depth:**

- An HTTP API has **routes** (a method plus a path), **integrations** (the target, here Lambda) and **stages** (a deployed version, here `$default`, which auto-deploys and adds no stage name to the URL). The invoke URL is the address the frontend calls.
- **Payload format 2.0** is how the request reaches Lambda. It carries the HTTP method, path parameters like `expenseId`, the body, and the authorizer's validated claims.
- The **JWT authorizer** needs two settings: the *issuer* (`https://cognito-idp.eu-west-2.amazonaws.com/<user pool ID>`, used to fetch Cognito's public keys and check the signature) and the *audience* (the app client ID, which must match the token's `client_id`). A mismatch returns 401 on every call. That is why the audience was updated when we created the new app client.
- The API must be given the **access token**, not the ID token, because the audience check relies on the access token's `client_id` claim.
- **CORS** is enforced by the browser. Before a call with an `Authorization` header, the browser sends an `OPTIONS` preflight asking whether the origin, headers and method are allowed. The API answers from its CORS settings. If they're wrong, the browser blocks the request and the app shows "could not reach the server". A trailing slash or a path in the allowed origin is a typical mistake.
- Postman can be used here to test the routes with a token before the frontend exists.

## Step 6: Frontend integration

**What we did:** built the frontend as plain HTML, CSS and JavaScript with four pages: a landing page, Dashboard, Expense History and Add Expense. Login and signup use Cognito's Hosted UI. `auth.js` sends users to Cognito, swaps the returned code for tokens (PKCE) and refreshes them. `api.js` calls the API with `Authorization: Bearer <access token>` and never sends a `userId`. The page scripts show the total, the category breakdown, the 5 most recent expenses, the history table with Delete, and the add form. Logout clears the tokens and ends the Cognito session. A `config.js` file holds the three environment values (Cognito domain, client ID, API URL), and only `config.example.js` goes to GitHub.

**Why:** the frontend is only the interface. All trust decisions stay in Cognito, API Gateway and Lambda. Using the Hosted UI cut the amount of custom code to what the brief asks for and removed a whole class of auth bugs. User-supplied text is inserted as plain text, so a description like `<b>x</b>` can't inject HTML.

**Role of the layer:** the browser code connects the user to the two backends we built, Cognito for identity and API Gateway for data, and presents the results.

**In depth:**

- The generated frontend only needs three environment values, kept in `config.js`: the Cognito domain, the app client ID and the API URL. Switching environments means changing those values, not the code.
- Integration is where the infrastructure meets the code. Each connection has a settings contract that must match on both sides: callback URL and scopes (Cognito and `auth.js`), token type and audience (`api.js` and the authorizer), allowed origin (CloudFront URL and CORS), and response field names (`expenseId`, `expenseDate` in Lambda and in the pages).
- Tokens are kept in the browser's local storage, and the access token is refreshed automatically with the refresh token when it expires (about an hour).
- If the API replies 401, the site signs the user out and returns to login. Errors from the API are shown to the user as short messages.
- Because the files are static, there is no build step. Deploying means copying the files to S3.

## Step 7: Deploy the frontend (S3, CloudFront, WAF)

**What we did:**

1. Created an S3 bucket in eu-west-2 with **Block all public access** left on, and uploaded `index.html`, `dashboard.html`, `expenses.html`, `add-expense.html` and the `css/` and `js/` folders at the bucket root.
2. Created a CloudFront distribution with the bucket as origin, secured with **Origin Access Control (OAC)**, default root object `index.html`, and HTTP redirected to HTTPS. We pasted CloudFront's generated bucket policy into S3 so only CloudFront can read the files.
3. Waited for the distribution to show Deployed and tested the public HTTPS URL.
4. Added the CloudFront URL to the Cognito callback and sign-out URLs and to the API Gateway CORS origins.
5. Attached AWS WAF to the distribution as an extra layer of protection (not required by the brief).
6. After any later file change: re-upload to S3 and create a CloudFront invalidation for `/*`.

**Why:** the bucket stays private, so the only way in is the CloudFront URL. That gives us HTTPS (a brief requirement), faster loading from edge caches, and no public S3 exposure. The URL is needed in Cognito and CORS, which is why this step had to come before the final end-to-end test.

**Role of the services:** Amazon S3 stores the static website files. Amazon CloudFront is the content delivery network that serves them securely and caches them. OAC lets CloudFront, and nothing else, read the private bucket. AWS WAF inspects incoming web requests and blocks malicious or abusive traffic (for example common exploit patterns or excessive request rates) before it reaches the site.

**In depth:**

- **Why not a public bucket or S3 website hosting:** that would serve plain HTTP and expose the bucket to the internet. With a private bucket and OAC, CloudFront signs its requests to S3 and the bucket policy allows only that distribution, so the S3 URL itself returns "access denied".
- **Default root object** `index.html` makes `https://<domain>/` return the landing page instead of an error.
- **Caching:** CloudFront keeps copies at edge locations. After uploading new files, the old copy may still be served until it expires, which is why we create an invalidation for `/*`. A page showing old CSS or an old `config.js` after an update is almost always cache.
- **Content types:** S3 stores each file's type (for example `text/css`). A wrong type makes the browser ignore a stylesheet, which is why pages can load without styling if files are uploaded incorrectly or in the wrong folder structure.
- **WAF** is a web application firewall made of a web ACL (rules) attached to the distribution. Typical rules are AWS managed rule groups for common attacks and a rate limit per IP. It is optional extra security beyond the brief and adds a small cost.
- The CloudFront domain (`*.cloudfront.net`) already comes with an HTTPS certificate, so no certificate setup is needed for this project.

## Step 8: Final integration and testing

**What we did:** tested the full flow on the deployed site and fixed what broke along the way. The two issues we hit are in the last section. Core features work on the live URL; the security checks below should be confirmed and screenshotted for the deliverables.

| Check | Status |
|---|---|
| Sign up, email verification, login | Working |
| Add expense, saved in DynamoDB | Working |
| Expense history and total | Working |
| Delete expense | Working |
| Logout | Confirm and screenshot |
| Invalid credentials rejected | Confirm and screenshot |
| API refuses requests without a token | Confirm and screenshot |
| User A cannot see or delete User B's expenses | Confirm with two accounts, screenshot |
| HTTPS and different browsers | Confirm and screenshot |

**Why:** each service worked alone, but the risks live in how they connect (callback URLs, CORS, authorizer audience, token types). Only a real end-to-end test on the public URL proves they line up, and the two-account test proves one user can't touch another's data.

**Role of CloudWatch:** Amazon CloudWatch stores the Lambda logs. When something fails behind the API, that's where the real error shows, while the browser only sees a generic message. Capture screenshots of each AWS resource and of the tests for the documentation deliverable.

**In depth:**

- **Two-user test:** create a second account in a private browser window, add an expense under each, and confirm each only sees its own. In DynamoDB the two items must carry different `userId` values, which are the two Cognito `sub` values.
- **No-token test:** call `GET /expenses` without an Authorization header (for example in Postman). The authorizer should answer 401 and Lambda should not run.
- **Ownership test:** a forged request can't set the owner, because Lambda ignores any `userId` in the body and uses the token's `sub`. Deleting an ID that belongs to someone else returns 404, the same as one that doesn't exist.
- **Where to look when something fails:** the browser console and Network tab show CORS, 401 and 4xx errors; CloudWatch Logs show Lambda errors; the Cognito and API Gateway settings show mismatches in URLs, scopes and audience.

- **Screenshots to capture:** DynamoDB table and items, Cognito pool and app client, Lambda code and role, API routes and authorizer, S3 bucket, CloudFront distribution and WAF, the live site, and each test result.

## Security measures and changes from the plan

**Security measures in place** (from section 11 of the brief):

- Passwords live only in Cognito, never in DynamoDB.
- Expense routes are protected by a Cognito JWT authorizer.
- The owner `userId` is the token's `sub`, taken by Lambda and never from the frontend.
- Every DynamoDB call is keyed on that `userId`, so users can't read or delete each other's expenses.
- Lambda has only the DynamoDB permissions it needs.
- No AWS credentials in the frontend and no secrets in GitHub (`config.js` is ignored; only `config.example.js` is committed).
- HTTPS is enforced by CloudFront, the S3 bucket is private behind OAC, and AWS WAF filters bad traffic.
- CORS only allows the CloudFront URL (and localhost for testing).

**What changed from the original plan:**

- The frontend team's custom login, signup, verify-email and forgot-password pages were dropped in favour of Cognito's Hosted UI, which is what the brief allows ("Cognito's managed authentication") and keeps the app simple. The Forgot Password feature comes from the Hosted UI at no extra code.
- The first app client had a client secret, so the token exchange failed with `invalid_client`. We created a new SPA client without a secret and updated `config.js` and the API Gateway authorizer audience.
- The first API calls failed with "could not reach the server" until CORS and the API URL were corrected.
- Categories are a field of an expense (the 6 in the brief), not a separate page. Everything outside the brief was removed.

**Still to deliver from the brief:** README, architecture diagram, screenshots of the AWS resources and tests, and the presentation.